

编译原理

复习课

中国海洋大学 计算机系 王欣捷

wangxinjie@ouc.edu.cn

考试详情

- 学期成绩评分体系：
  - 课下作业、课堂讨论及平常表现：20%
  - 实验课出勤及实验表现、实验报告成绩：20%
  - 期末考试成绩：60%

考试详情

- 期末考试共100分，涉及题型：

题型	分值
选择	10
填空	25
综合	65

考试详情

- 期末考试共100分，涉及1-7章内容：

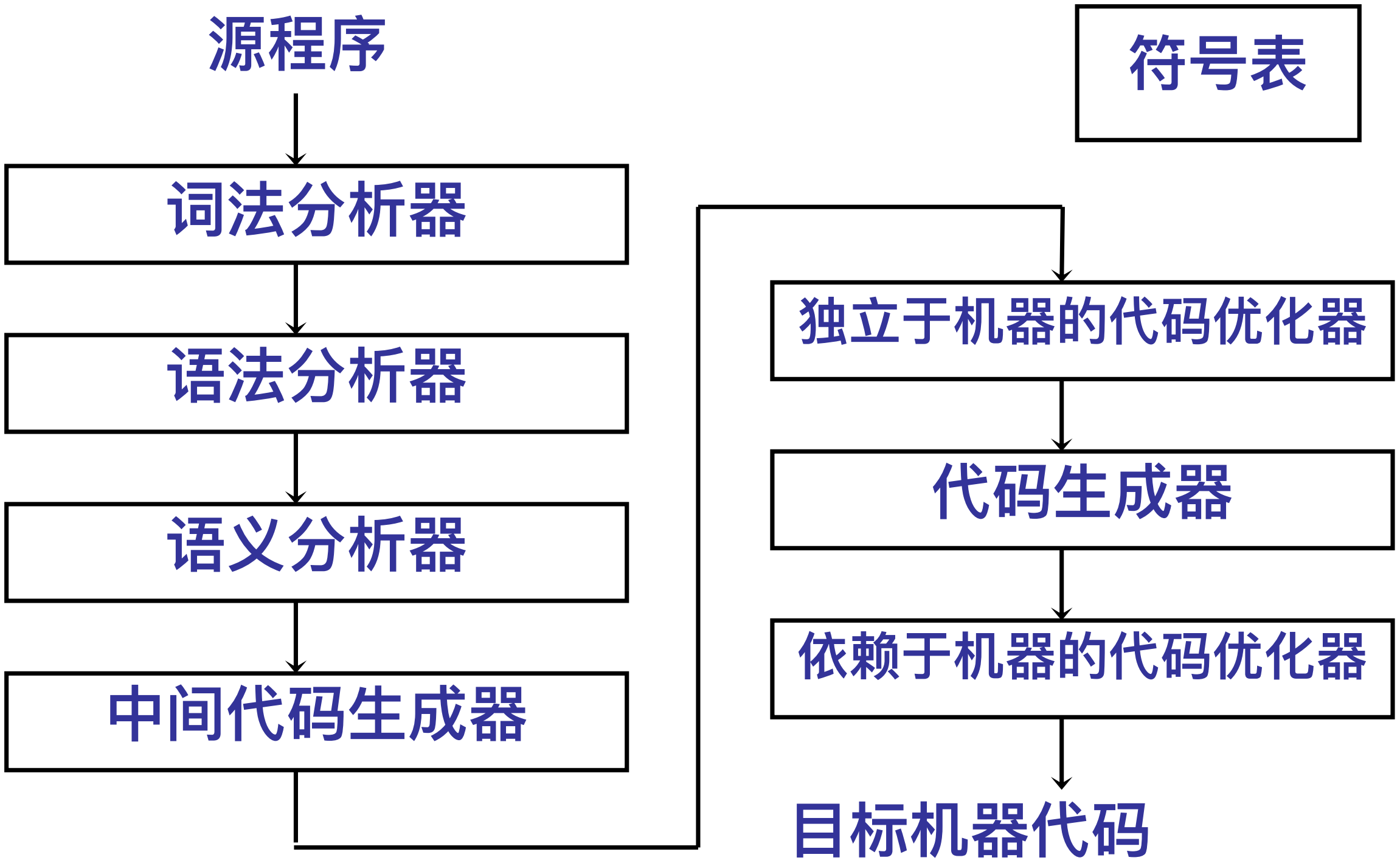
章节	约占比
第一章 概念 & 上机实验	2%~5%
第二章 词法分析	~20%
第三章 语法分析	~40%
第四章 语法制导的翻译	~10%
第五章 类型检查	~10%
第六章 运行时存储	~10%
第七章 中间代码生成	~10%

重点导航

- 第一章重点
- 第二章重点
- 第三章重点1、重点2、重点3
- 第四章重点
- 第五章重点
- 第六章重点
- 第七章重点

第一章 绪论

- **重点：编译器划分**



第一章 绪论

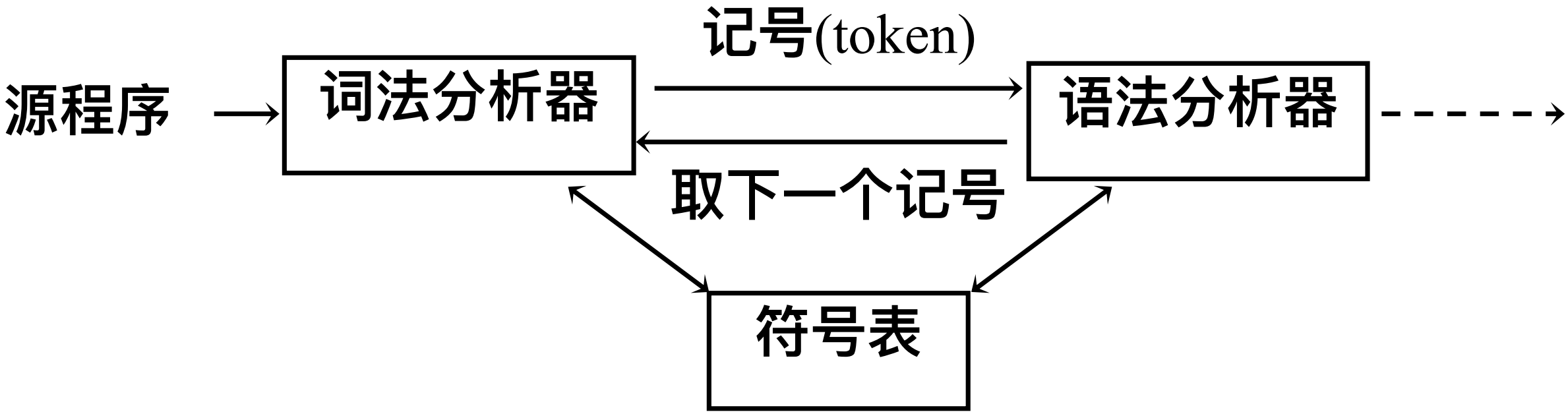
- 一些概念：
  - 翻译器、编译器和解释器
  - 源语言和目标语言
  - 交叉编译
- 编译程序的组织：
  - “遍”：对源程序或源程序的中间表示从头到尾扫描一次。
  - 编译阶段分组：前端和后端

第二章 词法分析

- **第二章线索：**
  - **词法分析器功能：字符流→记号流**
  - **词法分析器的实现：正规式→NFA→DFA→最简DFA→语言识别器。**
  - **词法分析的错误恢复。**

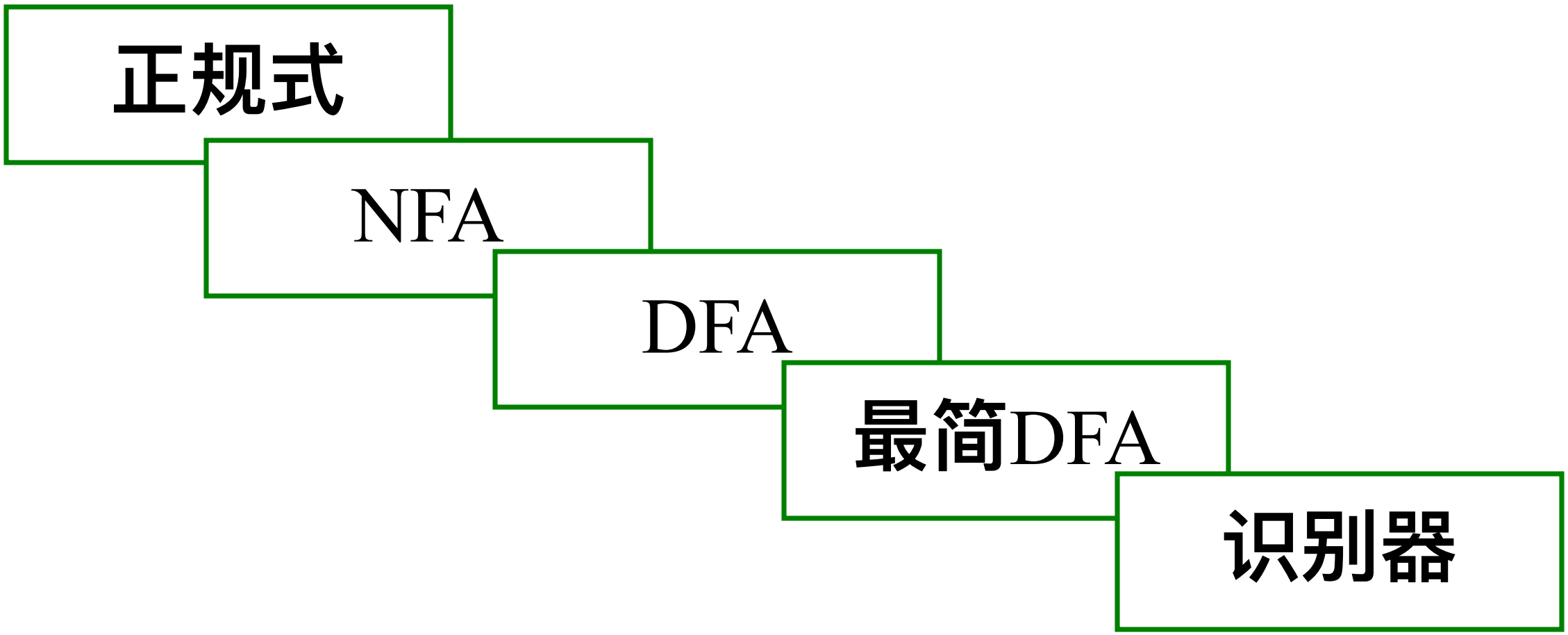
第二章 词法分析

- 词法分析功能：字符流→记号流
- 词法分析器和语法分析器的一种典型关系：语法分析器主导



第二章 词法分析

- 构造词法分析器（识别器）的过程：



第二章 词法分析

- 串起这些内容：
  - 词法分析的基本概念：串和语言，正规式等
  - 从非形式语言描述写出正规式或正规定义
  - NFA和DFA的概念
  - 从语言描述或正规式写出NFA（Thompson法）或DFA
  - NFA到DFA的转换：子集构造法
  - DFA的化简：基于转换函数是全函数的情况，否则需引入死状态。

第二章 词法分析

- 词法分析的主要任务：
  - 将正确的单词识别为记号
  - 删除分隔单词的空格（或制表符、回车符等分隔符）。删除注释。
  - 词法检查，报告错误。
- 词法记号（token）：<记号名，属性值>

第二章 词法分析

- 串和语言的相关概念和运算：
  - 字母表（通常用Σ表示）、串、语言、句子
  - 串的运算、语言的运算
- **正规式：**
  - 一种特殊表达式，是**模式**的形式表示，用于表示语言。
  - 正规式表示的语言称为**正规集**。

第二章 词法分析

正规式	正规集	备注
ε	{ε}	
a	$\{a\}$	$a \in \sum$
$(r) \mid (s)$	$L(r) \cup L(s)$	r和s是正规式
$(r)(s)$	$L(r)L(s)$	r和s是正规式
$(r)^*$	$(L(r))^*$	r是正规式
(r)	$L(r)$	r是正规式

第二章 词法分析

- 正规式的代数定律：可用于正规式的等价变换

$r\|s = s\|r$	|可交换
$r\|(s\|t) = (r\|s)\|t$	|可结合
$(rs)t = r(st)$	连接可结合
$r(s\|t) = rs\|rt;\ (s\|t)r = sr\|tr$	连接对|可分配
$\varepsilon r = r;\ r\varepsilon = r$	ε是连接的恒等元素
$r^* = (r\|\ \varepsilon)^* = \varepsilon\|rr^*$	ε肯定出现在一个闭包中
$r^{**} = r^*$	*是幂等的

第二章 词法分析

- 正规定义：为正规式命名
  - 例： Pascal语言的标识符集合：

$$\mathbf{letter} \rightarrow A \mid B \mid \ldots \mid Z \mid a \mid b \mid \ldots \mid z$$

$$\mathbf{digit} \rightarrow 0 \mid 1 \mid \ldots \mid 9$$

$$\mathbf{id} \rightarrow \mathbf{letter}(\mathbf{letter} \mid \mathbf{digit})^*$$

- 状态转换图：状态、边、初始状态、接受状态、动作等

第二章 词法分析

- **不确定的有限自动机（Nondeterministic Finite Automaton，简称NFA）是一个数学模型，它包括：**
  - **有限的状态集合S**
  - **输入符号集合∑（输入符号字母表）**
  - **转换函数$move : S \times (\sum \cup \{\varepsilon\}) \rightarrow P(S)$（$S$的幂集）**
  - **状态s_0是唯一开始状态**
  - **$F \subseteq S$是接受状态集合**

第二章 词法分析

- **确定的有限自动机**（Deterministic Finite Automaton，简称DFA）是一个数学模型，它包括：
  - 有限的状态集合S
  - 输入符号集合∑（输入符号字母表）
  - 转换函数$move: S \times \sum \rightarrow S$
  - 状态s_0是唯一开始状态
  - $F \subseteq S$是接受状态集合
- 与不确定有限自动机的区别：
  - 任何状态都没有ε转换
  - 最多有一条标记为a的边离开s

第二章 词法分析

- NFA和DFA可以用状态转换图或者状态转换表表示。
- NFA或DFA接受串*x*的定义
  - 当且仅当转换图中存在从开始状态到某个接受状态的路径，该路径各边上的标记拼成*x*。

第二章 词法分析

- NFA转换成等价DFA：子集构造法
  - 思路：将NFA的一个状态集合用DFA的一个状态代替；如果对NFA输入一个字符串$a_1 a_2 \ldots a_n$后，得到状态$s_1, s_2, \ldots, s_k$，则在DFA中用状态$\{s_1, s_2, \ldots, s_k\}$来代替这个状态集合。
  - 子集构造法中的一些运算：ε-*closure*(s)，ε-*closure*(T)，*move*(T,a)
  - 子集构造法不一定得到状态数最小的DFA

第二章 词法分析

- DFA的化简：
  - 该算法最小化DFA的状态数。
  - 该算法基于转换函数是全函数的情况，如果转换函数不是全函数，则先引入死状态转换成是全函数的情况。
  - 算法思路：把DFA状态分成一些不相交的子集，子集内的状态不可区别，不同子集的状态可区别。每个子集合并成一个状态。

第二章 词法分析

- **重点：掌握下列转换技巧、方法或算法**
  - 非形式描述的语言 ↔ 正规式（正规定义）
    - 词法分析(1)里的课堂练习、作业1、作业2、作业3、作业4、作业5
  - 正规式 → NFA（DFA、最简DFA）
    - 作业6、作业7、作业10
  - NFA → DFA
    - 作业8、作业9、词法分析(2)里的课堂练习
  - DFA → 最简DFA
    - 作业11、PPT中的2.3.4小节
  - 非形式描述的语言/正规式 ↔ NFA（DFA、最简DFA），即手工构造法
    - 词法分析(3)里的课堂练习和猎人过河
  - 简单的分析或证明
    - 例1、例2

第二章 词法分析

- 例1 (P39/2.17)： 若L是正规语言，证明下面L'也是正规语言。L'定义为：

  $L'=\{x \mid x^R \in L\}$，其中x^R表示x的逆。

思路：如果能构造出接受语言L'的NFA，就可以证明L'是正规语言。（因为NFA接受的是正规集）

返回

第二章 词法分析

- 例1解：

因为L是正规语言，因此必有一接受L的DFA M，可以基于M构造L’的NFA。

关键字：“反向”。

将M的所有边反向，M的开始状态变M’的接受状态，并新增一M’开始状态，令其有指向所有原M的接受状态的ε转换。

这样构造的M’就是接受L’的NFA

返回

第二章 词法分析

- 例2 (P39/2.18)：用C语言编译器编译下面的函数时，报告 parse error before ‘else’

```
long gcd(p,q)
long p,q;
{
    if (p%q == 0)
       /* then part */
       return q                    此处遗漏分号
  else
       /* else part */
       return gcd(q, p%q);
}
```

返回

第二章 词法分析

- 现在少了第一个注释的结束符号后，反而不报错了，请从词法分析角度分析为何此时不报错。

```
long gcd(p,q)
long p,q;
{
    if (p%q == 0)
       /* then part
       return q
  else
       /* else part */
       return gcd(q, p%q);
}
```

此处遗漏分号

返回

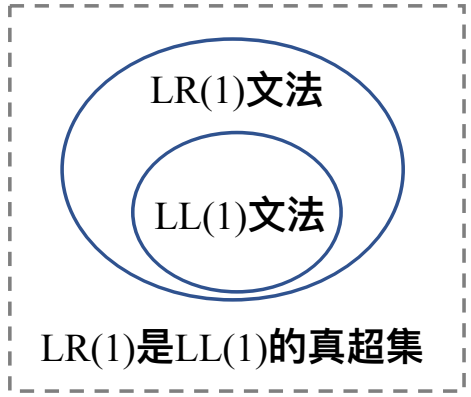
第三章 语法分析

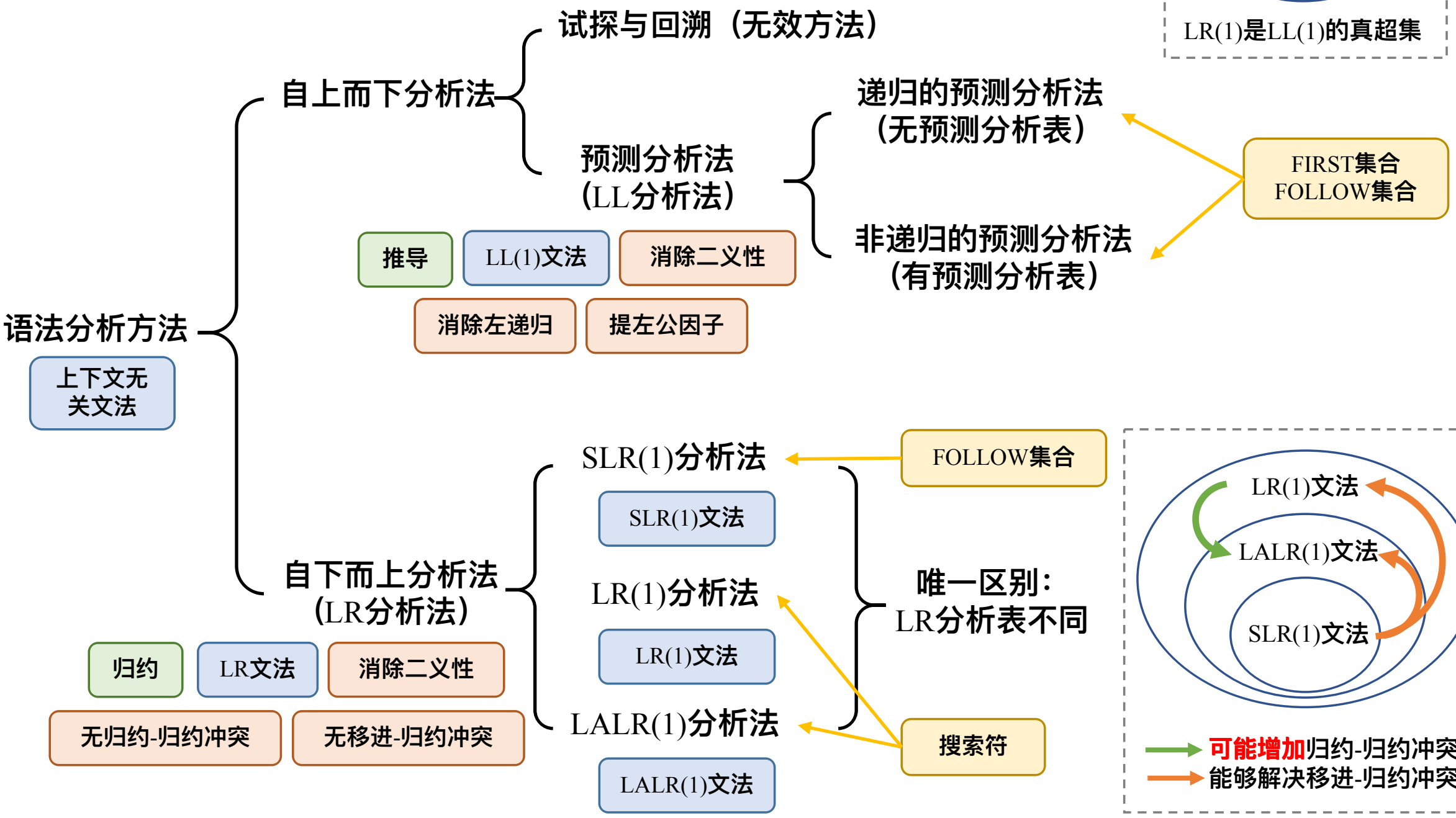
- 第三章线索：
  - 语法分析器功能：记号流→分析树（表示语法结构）
  - 上下文无关文法：
    - 推导、分析树和二义性
  - 自上而下分析：
    - LL(1)文法
    - 递归下降预测分析
    - 非递归预测分析（LL(1)分析）

第三章 语法分析

- **第三章线索：**
  - 自下而上分析
    - 移进-归约分析和句柄
    - LR分析法：SLR、规范LR（LR(1)）、LALR
    - 二义文法在LR分析中的应用
    - LR分析中的错误恢复

第三章 语法分析





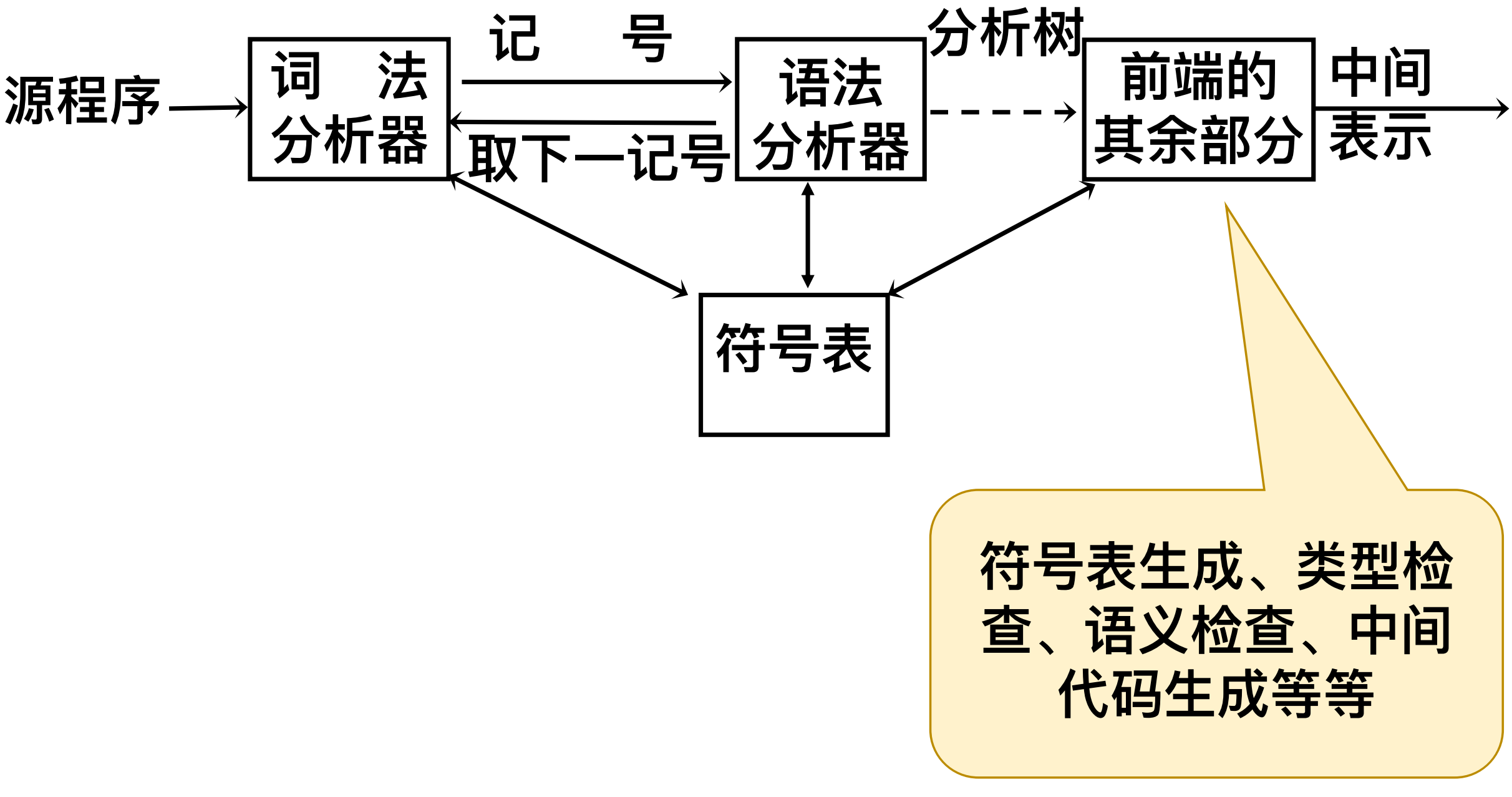
本章线索 思维导图

第三章 语法分析

- 语法分析： 记号流→分析树（表示语法结构）
  - 记号：二元组（记号名，属性值），由词法分析器生成。语法分析时可用记号名表示记号本身。
  - 语法结构：函数、表达式、语句等。可以用分析树来表示（概念上的，不一定真正生成）

第三章 语法分析

- 语法分析器在编译器模型中的位置：



第三章 语法分析

- 上下文无关文法G是四元组（V_T, V_N, S, P）
  - V_T: 终结符集合（非空，有限）
  - V_N: 非终结符集合（非空，有限）

    $V_T \cap V_N = \varnothing$
  - S: 开始符号
  - P: 产生式集合，产生式形式 ：$A \to \alpha$

    其中$A \in V_N$, $\alpha \in (V_T \cup V_N)^*$,

    S至少要出现在某产生式左部。
- 经过一些约定，上下文无关文法可仅用产生式集合表示。
- 一个经典的上下文无关文法的例子：算术表达式
  - $E \to E + E \mid E * E \mid (E) \mid \mathbf{id}$

第三章 语法分析

- 上下文无关文法：
  - **推导**：把产生式看成重写规则，把符号串中的非终结符用其产生式右部的串来代替
    - $\Rightarrow$， $\Rightarrow^*$， $\Rightarrow^+$
    - 最左推导$\Rightarrow_{lm}$，最右推导（规范推导）$\Rightarrow_{rm}$
    - 上下文无关语言，文法等价，句型（句子）、递归的概念
- 分析树：推导的图形表示。
  - 会画分析树

第三章 语法分析

- 上下文无关文法：
  - 分析树：推导的过程是一个由上而下建立分析树的过程。
  - 文法的二义性：文法存在某句子有不止一个最左（右）推导（即有两种不同分析树）。文法是二义的不一定表示该文法描述的语言是二义的。
  - Chomsky的文法分类： 0-3型文法（短语文法0⊃上下文有关文法1⊃上下文无关文法2⊃正规文法3）

第三章 语法分析

- 上下文无关文法：
  - 构造非二义文法：用优先级和结合性
  - 例：$E \rightarrow E+T \mid T$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid \mathbf{id}$$

第三章 语法分析

- 重点：上下文无关文法
  - 最左推导、最右推导、会画分析树、文法的二义性
    - 作业1、PPT中3.2.4/5小节
  - Chomsky的文法分类的基本概念
  - 会根据非形式语言描述设计上下文无关文法，会构造等价的二义和非二义文法
    - 作业2、作业3

第三章 语法分析

- 自上而下的分析：从文法的开始符号出发，自上而下，从左到右为输入串构造分析树。（是一种语法分析：输入串→分析树）
  - 一般方法：试探、回溯。
  - 如何消除回溯?
    - 无左递归（消除左递归）
    - 每次能选择正确的产生式（LL(1)文法）

第三章 语法分析

- 自上而下分析：
  - LL(1)文法定义：（需要理解并会求FIRST/FOLLOW集合）

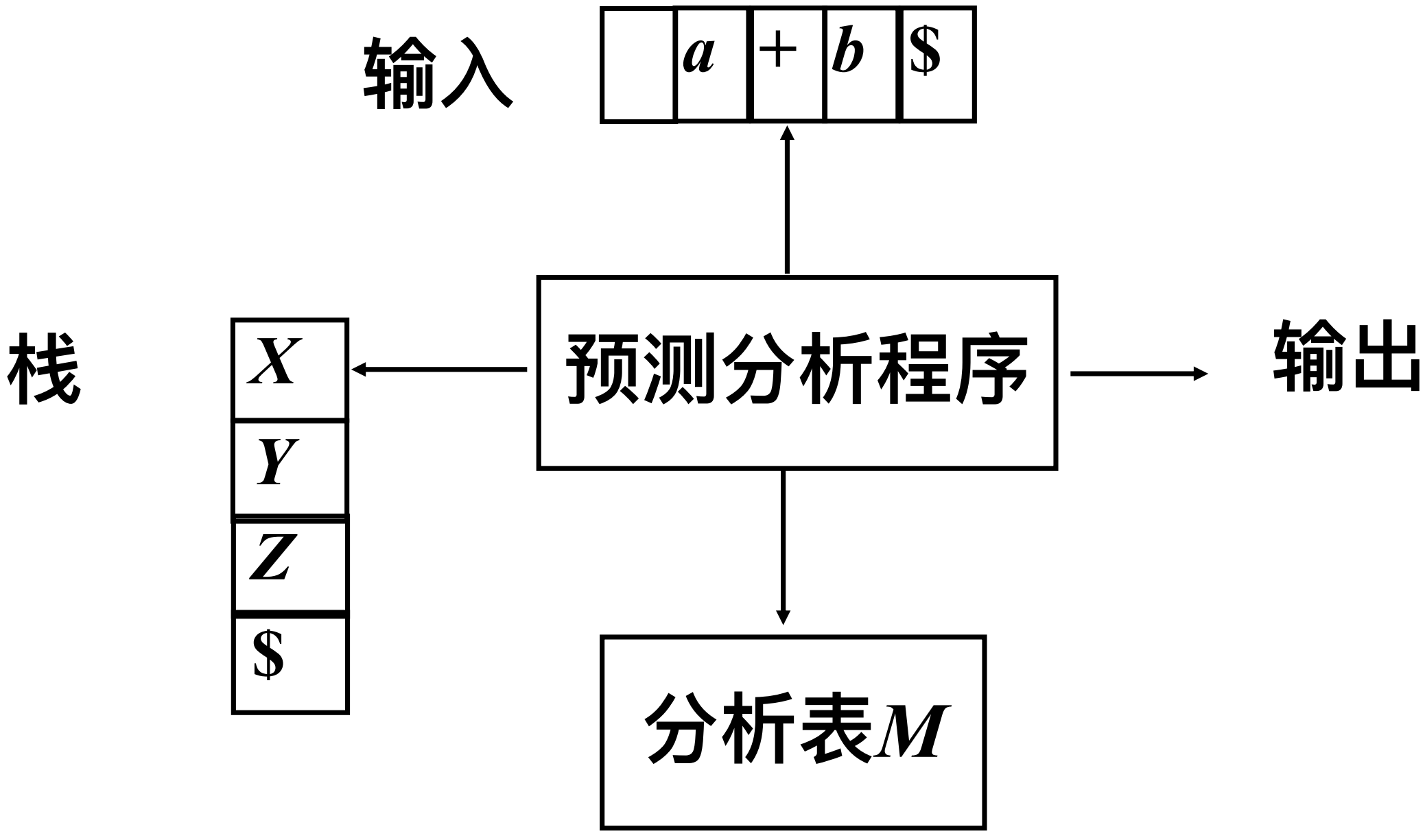
    任何两个产生式$A \to \alpha \mid \beta$都满足下列条件：
    - $\mathbf{FIRST}(\alpha) \cap \mathbf{FIRST}(\beta) = \varnothing$
    - 若$\beta \Rightarrow^* \varepsilon$，那么$\mathbf{FIRST}(\alpha) \cap \mathbf{FOLLOW}(A) = \varnothing$
  - 如果文法产生式有左因子，则肯定不满足上述第一个条件，因此需要消除左因子。

第三章 语法分析

- 自上而下的分析：
  - 对于LL(1)文法，怎样进行无回溯的自上而下分析？----递归下降的预测分析。
  - 对LL(1)文法，进行无回溯的自上而下分析，还可以用非递归预测分析。

第三章 语法分析

- 非递归的预测分析



第三章 语法分析

- 计算FIRST (*X*)：重复下列算法直到无新的终结符或ε可以加入FIRST (*X*)
  - 若*X*是终结符，则FIRST(*X*)={*X*}.
  - 若*X*→ε是产生式，则ε∈FIRST(*X*).
  - 若*X*→$Y_1Y_2\ldots Y_k$是产生式，则对某个*i*，若ε∈FIRST(Y_1)…FIRST(Y_{i-1}),而*a*∈FIRST(Y_i)，则*a*∈FIRST(*X*).若ε∈FIRST(Y_1)…FIRST(Y_k),则ε∈FIRST(*X*).
- 计算FIRST(*α*)时（*α*是文法符号串），参考上述*X*是产生式的情况。

第三章 语法分析

- 计算FOLLOW (A)：A是非终结符。

  重复下列算法直到无新的符号可以加入FOLLOW (A)。

  - 如果A是开始符号，则\$∈FOLLOW（A）
  - 若有产生式$B \rightarrow \alpha A \beta$，则FIRST($\beta$)中除ε以外的一切符号都属于FOLLOW($A$)
  - 若有产生式$B \rightarrow \alpha A$，或产生式$B \rightarrow \alpha A \beta$而ε∈FIRST($\beta$)，则FOLLOW($B$)⊆FOLLOW($A$)，即FOLLOW($B$)中的一切符号都要放入FOLLOW($A$)中。

第三章 语法分析

- 非递归的预测分析

预 测 分 析 表

非终结符	输入符号					
	id	+	*	(	)	$
E	$E\to TE'$			$E\to TE'$		
E'		$E'\to +TE'$			$E'\to\varepsilon$	$E'\to\varepsilon$
T	$T\to FT'$			$T\to FT'$		
T'		$T'\to\varepsilon$	$T'\to *FT'$		$T'\to\varepsilon$	$T'\to\varepsilon$
F	$F\to$id			$F\to(E)$		

第三章 语法分析

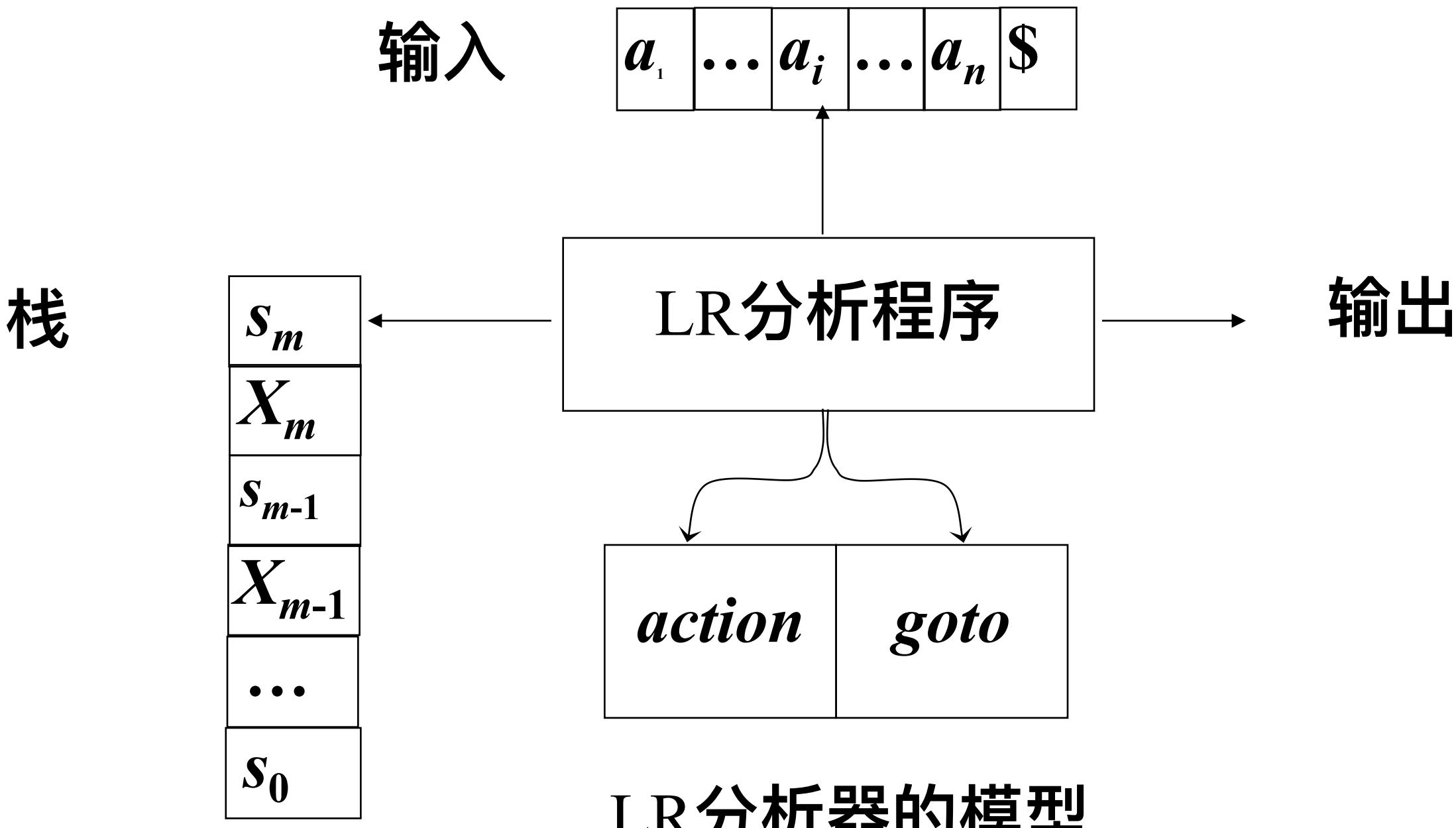
- 重点：自上而下的分析
  - 会消除左递归和提左因子
    - 作业4
  - LL(1)文法的定义，会计算FIRST和FOLLOW集合。
    - 作业5
  - LL(1)文法的自上而下分析：
    - 递归下降的预测分析：只需理解其原理
    - 非递归预测分析（LL(1)分析）：会构造LL(1)预测分析表，能够分析某句子的预测分析器动作。
      - 作业6、作业7

第三章 语法分析

- 自下而上的分析：移进-归约分析
  - 归约和句柄的概念。
  - 知道移进-归约分析的原理，以及移进-归约中的冲突。

第三章 语法分析

- LR分析原理



LR分析器的模型

第三章 语法分析

- LR(0)项目（简称项目）
  - 在右部的某个地方加点的产生式

$expr \rightarrow expr + \bullet\ term$

$term \rightarrow \bullet\ term * factor$

$term \rightarrow term \bullet * factor$

- 拓广文法

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \mathbf{id}$

第三章 语法分析

- 如何构造LR(0)项目集规范族?
  - *closure*(*I*)：如果*I*是文法*G*的一个项目集，那么*closure*(*I*)是用下面两条规则从*I*构造的项目集：

    (1) 初始时，*I*的每个项目都加入*closure*(*I*)。

    (2)如果$A\rightarrow\alpha\cdot B\beta$在*closure*(*I*)中，且 $B\rightarrow\gamma$ 是产生式，那么如果项目$B\rightarrow\cdot\gamma$还不在*closure*(*I*)中，则把它加入。重复该规则，直到没有更多项目可加入为止。
    - 核心项目和非核心项目
- 如何根据项目集规范族构造识别活前缀的DFA?

第三章 语法分析

- 构造SLR分析表的步骤：
  - 拓广文法，为产生式编号
  - 构造LR(0)项目集规范族
  - 根据项目集规范族构造识别活前缀的DFA。
  - 计算FOLLOW集合
  - 根据项目集族，DFA，FOLLOW集合和文法产生式构造SLR分析表。
- 构造规范LR分析表时要考虑搜索符（注意什么是搜索符，如何求搜索符），但是不需计算FOLLOW集合。
- 构造LALR分析表时还要合并同心集。合并同心集是否会产生新的移进-归约冲突或归约-归约冲突?

第三章 语法分析

- 重点：自下而上的分析
  - LR分析原理、归约、句柄等概念
    - 作业8
  - 会看LR分析表，弄懂分析某句子的LR分析步骤
  - 会三种构造LR分析表的方法：SLR、LR(1)、LALR
    - 作业9、作业11
  - 如何用二义文法构造无冲突的LR分析表：要会根据优先级和结合性消除分析表中的冲突条目
    - 作业14 （考试不会这么复杂）
  - 了解Yacc如何解决二义文法的冲突
  - 判断文法是否LL(1)、SLR、LR(1)、LALR(1)文法
    - 作业10、作业12、作业13

第四章 语法制导的翻译

- 第四章线索：
  - 语法制导定义、继承属性和综合属性、翻译方案
  - S属性定义的自下而上翻译

第四章 语法制导的翻译

- 重点：构造语法制导定义和翻译方案
  - 语法制导定义、综合属性、继承属性、翻译方案的概念，能区分综合属性和继承属性
  - S属性定义的概念
  - 能看懂构造抽象语法树的语法制导定义
  - 知道S属性定义如何自下而上计算
  - 会构造语法制导定义和注释分析树
    - 作业1、作业2
  - 会写翻译方案
    - 作业3

第五章 类型检查

- **第五章线索：**
  - 类型、执行错误、安全语言、类型化语言、类型系统等一系列概念。
  - 类型系统：类型表达式、断言、推理规则
  - 类型系统和类型检查器的例子
  - 笛卡尔积类型
  - 类型表达式的等价：结构等价、名字等价

第五章 类型检查

- **重点：类型检查**
  - 类型系统和类型检查器的例子：能看懂，理解相关类型表达式、断言、推理规则的含义
  - 会根据代码写类型表达式
    - 作业1
  - 类型表达式的等价：
    - 能弄清什么是结构等价、名字等价
    - 能弄清类型表示中的环是怎么回事，它对类型检查会有什么影响。

第六章 运行时存储空间的组织和管理

- **第六章线索：**
  - 过程、名字的作用域和绑定、活动记录、程序块等一系列概念
  - 活动记录的布局，局部数据的布局方式
  - 全局栈式存储分配时内存的划分
  - 活动树、控制栈和运行栈
  - 过程调用序列和返回序列
  - 参数传递：值调用、引用调用和换名调用

第六章 运行时存储空间的组织和管理

- **重点：活动记录、活动树、参数传递**
  - 活动记录的布局，局部数据的布局方式
  - 活动树、控制栈和运行栈
    - 运行时存储空间的组织和管理(1)的课堂练习(PPT44页)
  - 能看懂过程调用序列和返回序列
  - 参数传递：值调用、引用调用和换名调用
    - 作业1、作业2

第七章 中间代码生成

- 第七章线索：
  - 中间表示：后缀表示（逆波兰表示）、图形表示（抽象语法树和有向无环图）、三地址表示、静态单赋值形式
  - 各种语法结构如何翻译成上述中间表示。

第七章 中间代码生成

- 重点：
  - 后缀表示的定义
  - 表达式、赋值语句等如何翻译成后缀表示。
  - 图形表示：抽象语法树和有向无环图，表达式和赋值语句如何翻译成图形表示。
  - 会写三地址码
    - 作业1
  - 各语法结构如何翻译成三地址码
    - 教材P231 例7.2/7.3

总结

- 上课讲过的习题、课堂测试和课后布置的作业要会做。
- 上课做过的练习要掌握。
- 上课复习的重点要着重掌握。
- 考试中可能涉及2-5分的实验内容（考试中的实验部分没有在复习中体现，但上课肯定讲过，布置的实验也一定做过）
- 课件上星标章节不作为考试重点